

Java Map Methods Guide

HashMap · LinkedHashMap · TreeMap

All Methods · Code Examples · Interview Tips

■ Map Types in Java

Map Type	Ordering	Null Keys	Performance	Use When
HashMap	No order	1 null key allowed	O(1) get/put	Default choice, fastest
LinkedHashMap	Insertion order	1 null key allowed	O(1) get/put	Need insertion order
TreeMap	Sorted (natural)	No null keys	O(log n) get/put	Need sorted keys

How to create:

```
Map<String, Integer> map = new HashMap<>();
Map<String, Integer> lmap = new LinkedHashMap<>();
Map<String, Integer> tmap = new TreeMap<>();
```

■ 1. Core CRUD Methods

Method	Description	Returns	Example
put(K key, V value)	Insert or update key-value pair	(old value or null)	map.put("a", 1)
get(Object key)	Get value by key	V or null	map.get("a") → 1
getOrDefault(key, def)	Get value or default if missing	String default	map.getOrDefault("x", 0) → 0
remove(Object key)	Remove entry by key	V (removed value)	map.remove("a") → 1
remove(key, value)	Remove only if key maps to value	boolean	map.remove("a", 1) → true
containsKey(Object key)	Check if key exists	boolean	map.containsKey("a") → true
containsValue(Object val)	Check if value exists	boolean	map.containsValue(1) → true
size()	Number of key-value pairs	int	map.size() → 3
isEmpty()	Check if map is empty	boolean	map.isEmpty() → false
clear()	Remove all entries	void	map.clear()

■ Core Methods — Code Example

```
Map<String, Integer> map = new HashMap<>();

// put – insert
map.put("apple", 3);
map.put("banana", 5);
map.put("cherry", 2);

// get
System.out.println(map.get("apple")); // 3
System.out.println(map.get("mango")); // null

// getOrDefault – safe get (no NullPointerException)
System.out.println(map.getOrDefault("mango", 0)); // 0

// containsKey / containsValue
System.out.println(map.containsKey("banana")); // true
System.out.println(map.containsValue(99)); // false

// size / isEmpty
System.out.println(map.size()); // 3
System.out.println(map.isEmpty()); // false

// remove
map.remove("cherry");
System.out.println(map.size()); // 2

// clear
map.clear();
System.out.println(map.isEmpty()); // true
```

■ 2. Iteration Methods

Method	Description	Returns
keySet()	Get all keys	Set<K>
values()	Get all values	Collection<V>
entrySet()	Get all key-value pairs	Set<Map.Entry<K,V>>
forEach(BiConsumer)	Iterate with lambda	void

■ Iteration — Code Example

```
Map<String, Integer> map = new HashMap<>();
map.put("apple", 3);
map.put("banana", 5);
map.put("cherry", 2);

// 1. keySet() – iterate keys only
for (String key : map.keySet()) {
    System.out.println(key); // apple, banana, cherry
}

// 2. values() – iterate values only
for (int val : map.values()) {
    System.out.println(val); // 3, 5, 2
}

// 3. entrySet() – iterate key + value together (MOST USED)
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
    // apple = 3
    // banana = 5
    // cherry = 2
}

// 4. forEach with lambda – cleanest way
map.forEach((key, value) -> {
    System.out.println(key + " -> " + value);
});
```

■ **Interview Tip:** Always prefer `entrySet()` over iterating `keySet()` and calling `get(key)` inside — that causes two lookups per entry. `entrySet()` gives key and value in one shot.

■ 3. Compute & Merge Methods

Method	Description	Use Case
<code>putIfAbsent(key, val)</code>	Insert only if key not present	Initialize default values
<code>computeIfAbsent(key, fn)</code>	Compute & insert if key missing	Grouping / nested maps
<code>computeIfPresent(key, fn)</code>	Update value if key exists	Conditional update
<code>compute(key, fn)</code>	Compute value for key (always)	Frequency counting
<code>merge(key, val, fn)</code>	Merge new value with existing	Summing / combining values
<code>replace(key, val)</code>	Replace value for existing key	Update without insert
<code>replace(key, old, new)</code>	Replace only if current value matches	Safe conditional replace

■ Compute & Merge — Code Example

```

Map<String, Integer> freq = new HashMap<>();
String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};

// 1. putIfAbsent – insert only if key missing
freq.putIfAbsent("apple", 0);

// 2. compute – frequency counter (most common interview pattern)
for (String word : words) {
    freq.compute(word, (k, v) -> v == null ? 1 : v + 1);
}
System.out.println(freq); // {apple=3, banana=2, cherry=1}

// 3. getOrDefault + put – simpler frequency counter
Map<String, Integer> freq2 = new HashMap<>();
for (String word : words) {
    freq2.put(word, freq2.getOrDefault(word, 0) + 1);
}
System.out.println(freq2); // {apple=3, banana=2, cherry=1}

// 4. merge – combines values (same as compute but cleaner)
Map<String, Integer> freq3 = new HashMap<>();
for (String word : words) {
    freq3.merge(word, 1, Integer::sum);
}
System.out.println(freq3); // {apple=3, banana=2, cherry=1}

// 5. computeIfAbsent – grouping (group words by first letter)
Map<Character, List<String>> groups = new HashMap<>();
for (String word : words) {
    groups.computeIfAbsent(word.charAt(0), k -> new ArrayList<>()).add(word);
}
System.out.println(groups); // {a=[apple, apple, apple], b=[banana, banana], c=[cherry]}

// 6. replace
freq.replace("apple", 99);
System.out.println(freq.get("apple")); // 99

```

■ 4. Real Interview Coding Patterns

Pattern 1 — Frequency Counter

```
// Q: Count frequency of each character in a string
String s = "hello world";
Map<Character, Integer> freq = new HashMap<>();

for (char ch : s.toCharArray()) {
    freq.put(ch, freq.getOrDefault(ch, 0) + 1);
}
System.out.println(freq);
// {h=1, e=1, l=3, o=2, ' '=1, w=1, r=1, d=1}
```

Pattern 2 — Two Sum (index map)

```
// Q: Find two numbers that add up to target, return their indices
int[] nums = {2, 7, 11, 15};
int target = 9;
Map<Integer, Integer> map = new HashMap<>(); // value -> index

for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement)) {
        System.out.println(map.get(complement) + ", " + i); // 0, 1
        break;
    }
    map.put(nums[i], i);
}
```

Pattern 3 — Longest Substring Without Repeating

```
// Q: Find length of longest substring without repeating characters
String s = "abcabcbb";
Map<Character, Integer> map = new HashMap<>(); // char -> last seen index
int max = 0, l = 0;

for (int r = 0; r < s.length(); r++) {
    char c = s.charAt(r);
    if (map.containsKey(c)) {
        l = Math.max(l, map.get(c) + 1); // move left past duplicate
    }
    map.put(c, r);
    max = Math.max(max, r - l + 1);
}
System.out.println(max); // 3 ("abc")
```

Pattern 4 — Group Anagrams

```
// Q: Group words that are anagrams of each other
String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
Map<String, List<String>> map = new HashMap<>();

for (String word : words) {
    char[] chars = word.toCharArray();
    Arrays.sort(chars);
    String key = new String(chars); // sorted word as key
    map.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
}
System.out.println(map.values());
// [[eat, tea, ate], [tan, nat], [bat]]
```

Pattern 5 — First Non-Repeating Character

```
// Q: Find first character that appears only once
String s = "leetcode";
Map<Character, Integer> freq = new LinkedHashMap<>(); // preserves insertion order

for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}

for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
    if (entry.getValue() == 1) {
        System.out.println(entry.getKey()); // 'l'
        break;
    }
}
```

■ 5. Quick Reference — All Methods

Method	One-liner
<code>put(k, v)</code>	Insert or overwrite key
<code>get(k)</code>	Fetch value – returns null if missing
<code>getOrDefault(k, def)</code>	Fetch value – returns default if missing (safe)
<code>containsKey(k)</code>	Does key exist?
<code>containsValue(v)</code>	Does value exist? (O(n) scan)
<code>remove(k)</code>	Delete by key
<code>remove(k, v)</code>	Delete only if key maps to that exact value
<code>size()</code>	Total entries count
<code>isEmpty()</code>	Is the map empty?
<code>clear()</code>	Remove everything
<code>keySet()</code>	Set of all keys
<code>values()</code>	Collection of all values
<code>entrySet()</code>	Set of key-value Entry pairs
<code>forEach((k,v)->{})</code>	Lambda iteration – cleanest
<code>putIfAbsent(k, v)</code>	Insert only if key not already present
<code>replace(k, v)</code>	Update value for existing key only
<code>replace(k, old, new)</code>	Update only if current value matches old
<code>compute(k, (k,v)->{})</code>	Compute new value – null v means key was absent
<code>computeIfAbsent(k, fn)</code>	Compute and insert only if key missing
<code>computeIfPresent(k, fn)</code>	Compute and update only if key exists
<code>merge(k, v, fn)</code>	Combine new value with existing using function

■ Interview Quick Tips:

- Use `getOrDefault(key, 0)` for frequency counting — avoids null checks.
- Use `entrySet()` for iteration — single lookup vs `keySet() + get()`.
- Use `computeIfAbsent(key, k -> new ArrayList<>())` for grouping problems.
- Use `LinkedHashMap` when insertion order matters (first non-repeating char).
- Use `TreeMap` when sorted key order is needed.
- `HashMap` allows one null key; `TreeMap` does NOT allow null keys.